

Data structures

ASYMPTOTIC ANALYSIS & RECURRENCE

Today

1. Introduction to complexity analysis

Asymptotic:

- Big- O
- Big- Ω
- Θ

2. Code complexity analysis

3. Solving a recursive formula

Introduction to Complexity Analysis

Different algorithms can solve the same problem in different manners.

We need a way to estimate which one is better, how can we compare them?

The two most common complexity measures are:

- Time Complexity (run time)
- Space/Memory Complexity

We will focus mainly on worst-case time complexity:

- Worst case – worst possible input for an algorithm

Asymptotics

Approximation of the “growth ratio” of a function with respect to its input size.

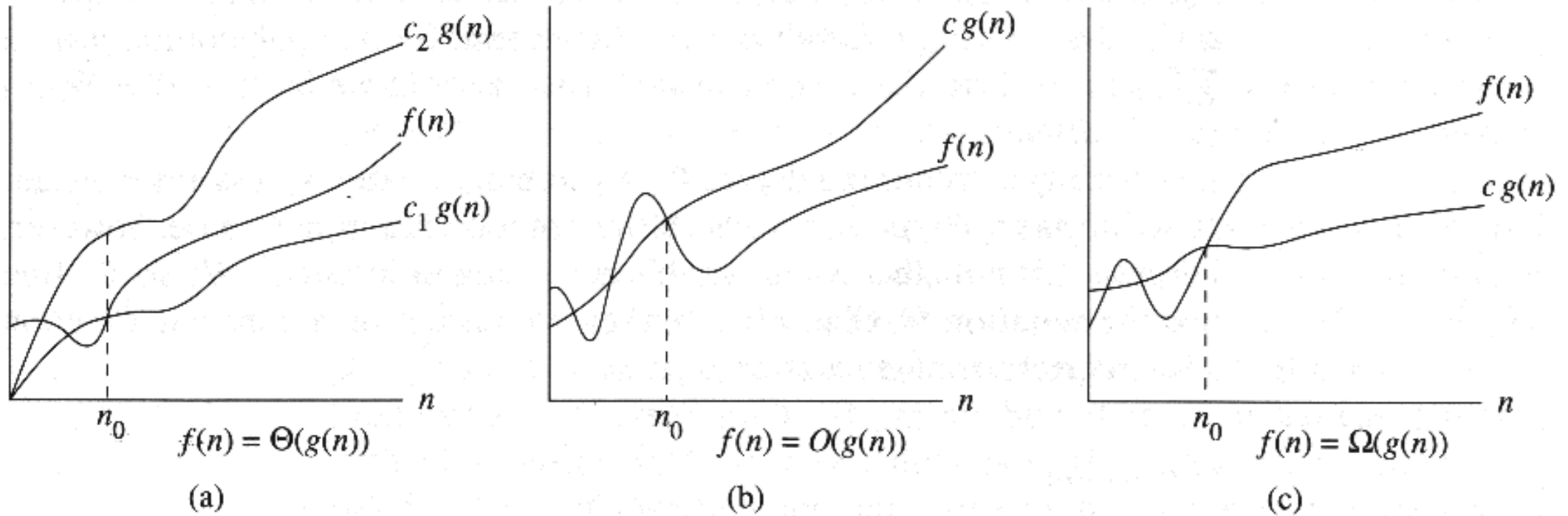
For two functions $f(n)$ and $g(n)$, we would like to determine if $f(n)$ is bigger, smaller or equal to $g(n)$ *asymptotically*.

Preliminary assumptions:

- A single machine with only one processor
- Simple operations are the done in run time of 1, such as:
 - Array cell access
 - Assignment
 - Arithmetical operations (+, -, *, /)
 - Boolean operations
 - Comparison

Asymptotics

For the following definitions we will assume we have an algorithm A that takes $f(n)$ operations to finish a task over input of size n .



Big-O notation

Big-O is used to denote an **upper** bound on a function.

$f(n) = O(g(n))$ means that when constant factors are ignored, then asymptotically $f(n)$ is "smaller or equal" to $g(n)$.

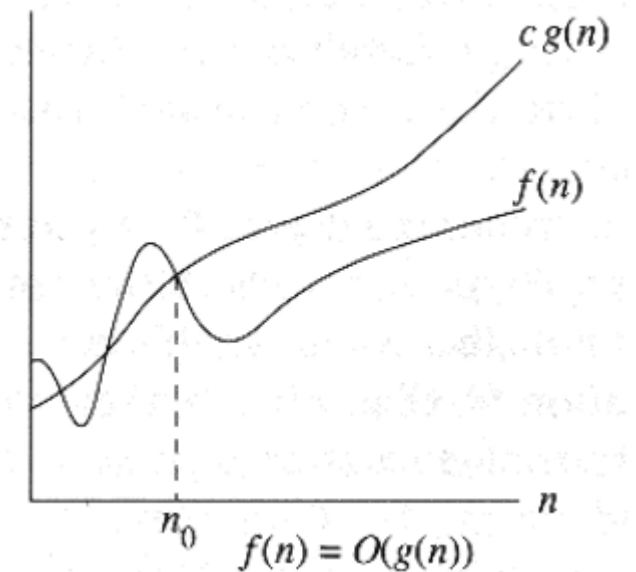
Definition :

$$f(n) \in O(g(n)) \Leftrightarrow$$

$$\exists c > 0, n_0 \geq 0$$

$$\text{s.t. } \forall n > n_0$$

$$\text{it holds that } f(n) \leq c \cdot g(n)$$



 Paul Bachmann introduced the O -notation in his book *Analytische Zahlentheorie* (1894)

Exercise 1

- Let $f(n) = n \log(n^5) + 6n$. Prove that $f(n) = O(n \log n)$, and $f(n) \neq O(n)$.

Exercise 1

- Let $f(n) = n \log(n^5) + 6n$, Prove that $f(n) = O(n \log n)$, and $f(n) \neq O(n)$.

Solution:

- We'll find $c > 0, n_0 \geq 0$ such that for any $n > n_0$, $f(n) \leq cn \log(n)$.
- For any $n \geq 2$:
$$f(n) = n \log(n^5) + 6n = 5n \log(n) + 6n \leq 5n \log(n) + 6n \log(n) = 11n \log(n).$$
- So, for $n_0 = 2$, and $c = 11$, it holds that for any $n > n_0$, $f(n) \leq cn \log(n)$

Exercise 1

- Let $f(n) = n \log(n^5) + 6n$, Prove that $f(n) = O(n \log n)$, and $f(n) \neq O(n)$.

Solution:

- To see that $f(n) \neq O(n)$, suppose towards a contradiction there exists $c > 0, n_0 \geq 0$ such that for any $n > n_0, f(n) \leq cn$.
- In this case, we will show there exists $N \geq n_0$ such that $f(N) > cN$.
- Indeed, if $f(n) \leq cn$, and $n \geq 2$, this implies in particular: $n \log(n) \leq cn$, which is equivalent to $\log(n) \leq c$.
- But, clearly, if $n > 2^c, \log(n) > c$. So, if $N = \max(n_0, 2^c) + 1$, then $N > n_0$ and $f(N) > cN$.
- It is important to note here, that the function $\log(n)$ is unbounded, so it can never be true that for some constant $c > 0$, it holds that $\log(n) \leq c$.

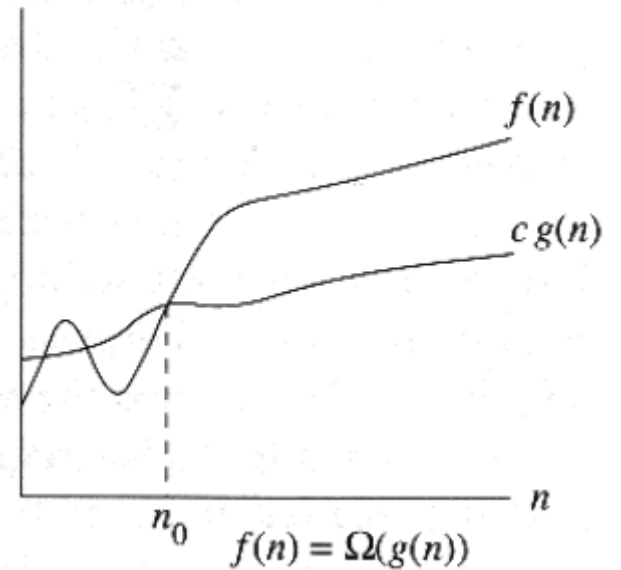
Big- Ω notation

Big- Ω is used to denote a **lower** bound of an algorithm's time complexity.

$f(n) = \Omega(g(n))$ means that when constant factors are ignored, then asymptotically $f(n)$ is "greater or equal" to $g(n)$.

Definition:

$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c > 0, n_0 \geq 0 \text{ s.t. } \forall n > n_0 \text{ it holds that}$
 $0 \leq c \cdot g(n) \leq f(n)$



Exercise 2

- Let $f(n) = 2^n$, prove that $f(n) \in \Omega(3^{\sqrt{n}})$.

Solution:

- We'll find $c > 0, n_0 \geq 0$ such that for any $n > n_0$, $f(n) \geq c \cdot 3^{\sqrt{n}}$.
- It is more convenient to consider the logarithms of the functions.
- Note that $4 > (\log(3))^2$, therefore, for any $n \geq 4$: $\sqrt{n} \geq \log(3)$.
- $\log(f(n)) = n = \sqrt{n}\sqrt{n} \geq \sqrt{n} \log(3) = \log(3^{\sqrt{n}})$.
- Since, the logarithm function is monotone, this implies $f(n) \geq 3^{\sqrt{n}}$.
- So, for $n_0 = 4, c = 1$, it holds that for any $n > n_0$, $f(n) \geq 3^{\sqrt{n}}$.

θ notation

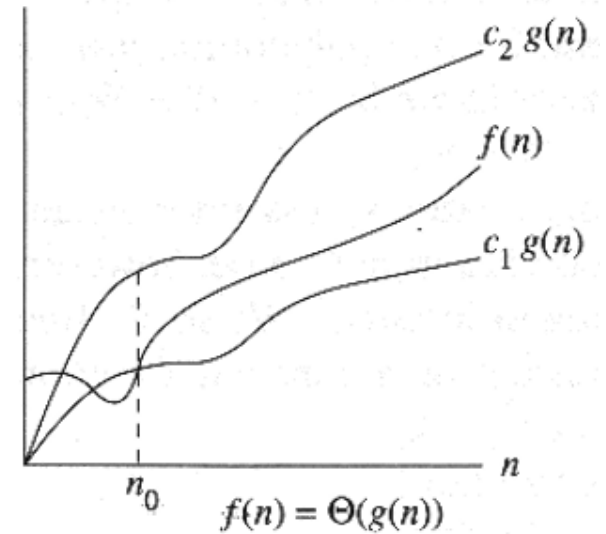
θ holds when both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ holds.

$f(n) = \theta(g(n))$ means that when constant factors are ignored, then asymptotically $f(n)$ is "equal" to $g(n)$.

Definition:

$f(n) \in \theta(g(n)) \Leftrightarrow \exists c_1, c_2 > 0, n_0 \geq 0$ s. t. $\forall n > n_0$, it holds that

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$



Exercise 3

Let $f(n) = \log(n!)$, prove that $f(n) \in \theta(n \log n)$.

Solution:

- To show that $\log(n!) = O(n \log n)$ we note that
 - $\log(n!) = \sum_{i=1}^n \log(i) \leq n \log(n)$, so $n_0 = 2$, and $c = 1$ works.
- To show that $\log n! = \Omega(n \log n)$ we note that
 - $\log(n!) = \sum_{i=1}^n \log(i) \geq \sum_{i=n/2}^n \log(i) \geq \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{1}{2}n \log(n) - \frac{1}{2}n \log(2)$
 $\geq \frac{1}{4}n \log(n)$, where the last inequality is true for every $n > 4$.
 - so $n_0 = 4$, and $c = 0.25$ works.

Exercise 4 (prove or disprove)

Exercise :

- For every two functions f and g , either $f = O(g)$ or $g = O(f)$.

Exercise 4 Solution

Exercise :

- For every two functions f and g , either $f = O(g)$ or $g = O(f)$.

Solution:

- **False.** Consider

$$f(n) = n$$
$$g(n) = \begin{cases} n^2 & \text{if } n \text{ is even} \\ 1 & \text{if } n \text{ is odd} \end{cases}$$

Short Summary

$$f(n) = O(g(n)) \approx f(n) \text{ “} \leq \text{” } g(n)$$

$$f(n) = \Omega(g(n)) \approx f(n) \text{ “} \geq \text{” } g(n)$$

$$f(n) = \theta(g(n)) \approx f(n) \text{ “} = \text{” } g(n)$$



NOTE: the signs $\leq$, $\geq$, $=$, $<$, $>$ are just for intuition, and **not** real equations.

Do not use these notations.

Special Properties

Transitive

- If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$
- If $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$, then $f(n) = \Omega(h(n))$
- If $f(n) = \theta(g(n))$ and $g(n) = \theta(h(n))$, then $f(n) = \theta(h(n))$

Reflexive

- $f(n) = O(f(n))$
- $f(n) = \Omega(f(n))$
- $f(n) = \theta(f(n))$

Symmetric

- $f(n) = \theta(g(n))$ iff $g(n) = \theta(f(n))$

NOTE: O and Ω are not symmetric.

Complexity Analysis of Recursive Algorithms

Recursion defines the solution to a problem using the solution to a smaller instance of the problem.

Examples:

- Computing factorial

- $n! = (n-1)! * n$

- Fibonacci series

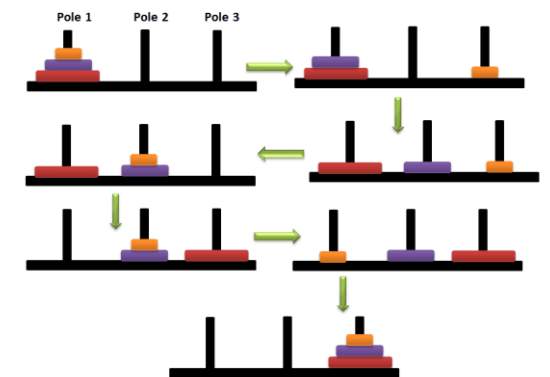
- $f(n) = f(n-1) + f(n-2)$

- Solving Towers of Hanoi

To move n rings: move $n-1$ rings to aux pole,
move 1 ring, then move $n-1$ rings from aux pole

- Binary Search

0, 1, 1, 2, 3, 5, 8, 13, ...



How to Analyze Recursive Algorithms?

We want to find the running time $T(n)$ of our algorithm

1. Analyze the pseudocode and identify a recurrence relation for $T(n)$.
2. Solve the recurrence relation:
 - Use the substitution method.
 - Or prove by induction.
 - Another approach is also available to solve this problem.

Exercise 5

What is the runtime complexity of the following pseudocode?

```
My_algorithm(n)
  If n>1 then
    For each i from 1 to n-1
      call function Do_something()
    Let m = floor(n / 2)
    Call function My_algorithm(m)
```

*Assume the runtime of Do_something is $O(1)$

Exercise 5

Given the following code:

```
My_algorithm(n)
  If n > 1 then
    For each i from 1 to n - 1
      call function Do_something()
    Let m = floor(n / 2)
    Call function My_algorithm(m)
```

Assume that the runtime of Do_something is $O(1)$.

- 1 – Find a recursive formula $T(n)$ for the runtime complexity of the given code.**
- 2 – Find a closed form solution of $T(n)$ using the substitution method.**

Exercise 5

Given the following code:

My_algorithm(n)	T(n)=
If n > 1 then	O(1)
For each i from 1 to n - 1	O(n)
call function Do_something()	· O(1)
Let m = floor(n / 2)	O(1)
Call function My_algorithm(m)	T(m) = T(n/2)

$T(n) =$

Exercise 5

Given the following code:

My_algorithm(n)	T(n)=
If n > 1 then	O(1)
For each i from 1 to n - 1	O(n)
call function Do_something()	· O(1)
Let m = floor(n / 2)	O(1)
Call function My_algorithm(m)	T(m) = T(n/2)

$$T(n) = O(1) + O(n \cdot 1) + O(1) + T\left(\frac{n}{2}\right)$$

Exercise 5

Given the following code:

My_algorithm(n)	T(n)=
If n > 1 then	O(1)
For each i from 1 to n - 1	O(n)
call function Do_something()	· O(1)
Let m = floor(n / 2)	O(1)
Call function My_algorithm(m)	T(m) = T(n/2)

$$T(n) = O(1) + O(n \cdot 1) + O(1) + T\left(\frac{n}{2}\right) \leq T\left(\frac{n}{2}\right) + c_2 n$$

Exercise 5

Given the following code:

My_algorithm(n)	T(n)=
If n > 1 then	O(1)
For each i from 1 to n - 1	O(n)
call function Do_something()	· O(1)
Let m = floor(n / 2)	O(1)
Call function My_algorithm(m)	T(m) = T(n/2)

$$T(n) = O(1) + O(n \cdot 1) + O(1) + T\left(\frac{n}{2}\right) \leq T\left(\frac{n}{2}\right) + c_2 n$$

$$T(1) = c_1$$

Exercise 5

Given the following code:

My_algorithm(n)	T(n)=
If n > 1 then	O(1)
For each i from 1 to n - 1	O(n)
call function Do_something()	· O(1)
Let m = floor(n / 2)	O(1)
Call function My_algorithm(m)	T(m) = T(n/2)

Recursive formula: $T(n) = O(1) + O(n \cdot 1) + O(1) + T\left(\frac{n}{2}\right) \leq T\left(\frac{n}{2}\right) + c_2 n$

Base case: $T(1) = c_1$

How to Solve Recurrence Relations?

The iteration/substitution method

- Assign the formula into itself several times
- Derive the general form obtained after substituting k times
- Compute how many substitutions are required until reaching the base case
- Compute the resulting sum
- Assign an initial condition and extract a closed form solution
- Prove your guess by induction

Exercise 5

$$T(n) \leq T\left(\frac{n}{2}\right) + c_2 n$$
$$T(1) = c_1$$

```
My_algorithm(n)
  If n > 1 then
    For each i from 1 to n - 1
      call function Do_something()
    Let m = floor(n / 2)
    Call function My_algorithm(m)
```

- Assign the formula into itself several times

$$T(n) \leq T\left(\frac{n}{2}\right) + c_2 n \leq T\left(\frac{n}{4}\right) + c_2 \frac{n}{2} + c_2 n \leq T\left(\frac{n}{8}\right) + c_2 \frac{n}{4} + c_2 \frac{n}{2} + c_2 n$$
$$\leq \dots \leq T\left(\frac{n}{2^k}\right) + c_2 \frac{n}{2^{k-1}} + \dots + c_2 \frac{n}{2} + c_2 n = T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} c_2 \frac{n}{2^i}$$

- Make an educated “guess”

$$T(n) = O(n)$$

- Prove your guess by induction

- We will prove by induction, that there exists a constant $c > 0$, such that for any $n \geq 1$:

$$T(n) \leq c \cdot n$$

- How can we find that constant?

Exercise 5

$$T(n) \leq T\left(\frac{n}{2}\right) + c_2 n$$
$$T(1) = c_1$$

```
My_algorithm(n)
  If n > 1 then
    For each i from 1 to n - 1
      call function Do_something()
    Let m = floor(n / 2)
    Call function My_algorithm(m)
```

- Assign the formula into itself several times

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{2}\right) + c_2 n \leq T\left(\frac{n}{4}\right) + c_2 \frac{n}{2} + c_2 n \leq T\left(\frac{n}{8}\right) + c_2 \frac{n}{4} + c_2 \frac{n}{2} + c_2 n \\ &\leq \dots \leq T\left(\frac{n}{2^k}\right) + c_2 \frac{n}{2^{k-1}} + \dots + c_2 \frac{n}{2} + c_2 n = T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} c_2 \frac{n}{2^i} \end{aligned}$$

- What should k be so that $\frac{n}{2^k} = 1$ (the base case)?
 $k = \log n$
- $T(n) \leq T(1) + \sum_{i=0}^{\log n - 1} c_2 \frac{n}{2^i} \leq c_1 + 2c_2 n = O(n)$

$$T(n) \leq T\left(\frac{n}{2}\right) + c_2 n$$

$$T(1) = c_1$$

Exercise 5

Proving the same bound using induction:

- Note that the definition of $T(n)$ hides two constants.

$$T(1) = c_1 \text{ and } T(n) \leq T\left(\frac{n}{2}\right) + c_2 n$$

- To satisfy both $T(1)$ and $T(n)$, we'll use $c = \max(c_1, 2c_2)$.
- We now show $T(n) \leq cn$, for $n \geq 1$.
- **Basis:** $n = 1$. In this case, $T(1) = c_1 \leq \max(c_1, 2c_2) = c$.
- **Assumption:** Assume that for $k < n$, $T(k) \leq ck$ and prove $T(n) \leq cn$.
- **Step:** Using the induction assumption:

$$T(n) \leq T\left(\frac{n}{2}\right) + c_2 n \leq_1 c \frac{n}{2} + c_2 n = \frac{cn + 2c_2 n}{2} \leq_2 \frac{cn + \max(c_1, 2c_2) n}{2} = \frac{cn + cn}{2} = cn$$

Exercise 6

True/False:

There exist functions $f(n)$ and $g(n)$ such that for all $n > 0$, $g(n) < f(n)$ and $f(n) = O(g(n))$.

Solution: True

Consider $f(n) = 2n + 1$, $g(n) = n$.

For all $n > 0$: $n < 2n + 1$ and $2n + 1 = O(n)$

Exercise 7 (Moed A 2022)

Let B be an algorithm. Denote by $T_A(n)$ the average-case time complexity of B , and by $T_W(n)$ its worst-case time complexity, on an input of size n . Suppose that $T_A(n) \in \Theta(n^2 \log n)$.

True / False: The worst-case must satisfy $T_W(n) \in \Omega(n \log n)$.

ANSWER: True.

By definition of average and worst-case complexity, we have $T_W(n) \geq T_A(n)$.

By the definition of $\Theta(\cdot)$, we have that $\exists c > 0$, and n_0 , such that $T_A(n) \geq c n^2 \log n$ for $n \geq n_0$.

It follows that for $T_W(n) \geq c n^2 \log n$ for $n \geq n_0$. Since $n^2 \geq n$ for $n \geq 1$, it follows that $T_W(n) \geq c n \log n$.